SARDAR PATEL INSTITUTE OF TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACADEMIC YEAR 2026–27 | B.TECH THIRD YEAR (SEMESTER V)

Project Title: PERSISTENT EVENT-DRIVEN MEMORY CACHE MODULE	**Project Guide:** Prof. Suhas Kakade
Team Members: 1. Jay Shinde (2024800115) 2. Nikhil Shelar (2024800113) 3. Sanchit Gaikwad (2024800029)	**Document Scope:** Complete 12-Week Semester Implementation Logbook (Phase I through Final ESE) **Verification:** Verified against repository tests and deliverables

WEEK 1 (28 Jul 2026 – 03 Aug 2026) — Phase I: Problem Definition & Inception

Core Objective & Focus: Topic Selection, RAM Volatility Research, Core Problem Formulation & Architecture Modeling

Detailed Work Undertaken & Key Technical Achievements:

- Finalized project title 'Persistent Event-Driven Memory Cache Module' and registered under Prof. Suhas Kakade.
- Researched the fundamental systems trade-off: RAM microsecond access speed vs. volatile data loss on crash/reboot.
- Formulated central research question: 'How can we combine RAM-level speed with persistence, recovery and event-driven monitoring?'.
- Identified the 7 core pitfalls in naive in-memory collections (volatility, state loss, cold start stampedes, torn updates, zero observability, memory leaks, and concurrency hazards).
- Drafted initial 4-tier conceptual architecture (Client -> Facade -> Engine -> [WAL / Bus / Eviction / Concurrency]).
- Divided core module ownership among team members based on technical specialization.

Individual Team Member Contributions:

- **Jay Shinde:** Analyzed in-memory key-value hashing structures and memory-access performance benchmarks.
- **Nikhil Shelar:** Researched pub-sub event architectures and event notification decoupled pipelines.
- **Sanchit Gaikwad:** Studied database Write-Ahead Logging (WAL) theory and crash recovery protocols.

Key Technical Deliverable: Draft Problem Statement, Initial Project Scope Document, and Conceptual Architecture Model.	**Evaluation Status:** **COMPLETED & VERIFIED**

WEEK 2 (04 Aug 2026 – 10 Aug 2026) — Phase I: Literature Survey & Gap Analysis

Core Objective & Focus: Comparative Study of Related Approaches, 5-Way Gap Analysis & Technology Selection

Detailed Work Undertaken & Key Technical Achievements:

- Conducted literature survey comparing Java HashMap, Guava/Caffeine caches, and RocksDB/LevelDB persistent stores.
- Established academic positioning disclaimer (educational systems synthesis, not replacing production Redis/DBs).
- Formulated the 5-way engineering gap: Fast Memory + WAL Durability + Event Bus + TTL/Eviction + Concurrency Safety.
- Selected pure Java 17 with standard JDK libraries (zero third-party dependencies) for complete white-box transparency.
- Designed the Write-Ahead Log (WAL) record binary structure with 64-bit sequence numbers and CRC32 checksums.
- Defined the 'WAL-before-memory' invariant rule ensuring all updates are durable prior to memory modification.

Individual Team Member Contributions:

- **Jay Shinde:** Formalized comparative taxonomy table and state mutation concurrency specifications.
- **Nikhil Shelar:** Designed event envelope specification (`CacheEvent`) and telemetry metric counters.
- **Sanchit Gaikwad:** Designed WAL sequential binary serializer and POSIX file lock exclusion mechanism.

Key Technical Deliverable: Literature Survey Matrix, Gap Analysis Report, and WAL Specification Draft.

Evaluation Status: **COMPLETED & VERIFIED**

WEEK 3 (11 Aug 2026 – 17 Aug 2026) — Phase I Review & Baseline Prototype

Core Objective & Focus: Phase-I Defense, Module Interface Definition & Foundational Scaffolding

Detailed Work Undertaken & Key Technical Achievements:

- Successfully presented Phase-I presentation and technical report to guide Prof. Suhas Kakade.
- Finalized semester milestone roadmap (Phase I to Final ESE) and weekly evaluation schedule.
- Created clean project repository scaffolding with GNU Makefile and Maven build targets.
- Developed baseline data structures: `CacheEntry` with version counters, `LookupResult`, and `CacheConfig`.
- Implemented baseline `Phase1CLI` supporting interactive `SET`, `GET`, `DELETE`, and `CRASH` commands.
- Established unit testing harness (`TestSupport.java`) with isolated temporary directory management.

Individual Team Member Contributions:

- **Jay Shinde:** Created `CacheEntry` model, timestamp validation, and Phase-I presentation slide deck.
- **Nikhil Shelar:** Authored `Phase1CLI` command parser and automated test verification harness.
- **Sanchit Gaikwad:** Implemented initial disk directory manager and baseline log file writer.

Key Technical Deliverable: Phase-I Signed Report, `phase-1/` documentation repository, and baseline CLI prototype.

Evaluation Status: **COMPLETED & VERIFIED**

WEEK 4 (18 Aug 2026 – 24 Aug 2026) — Phase II: Core Engine & Memory Subsystem

Core Objective & Focus: Thread-Safe CacheEngine, Version Tracking & Invariant Verification

Detailed Work Undertaken & Key Technical Achievements:

- Implemented `CacheEngine.java` using concurrent hash map storage with nanosecond lookup performance.
- Implemented monotonic entry version tracking (`version = 1, 2, ...`) for every key update.
- Built `InvariantReport.java` verifying internal state consistency between storage map and metadata.
- Integrated lazy expiration evaluation on `get()` operations (checking `now > expiresAt`).
- Implemented Compare-And-Set (CAS) atomic primitives (`cas(key, expectedVer, newValue)`).
- Added unit tests covering entry overwrites, deletion tombstones, and concurrent thread updates.

Individual Team Member Contributions:

- **Jay Shinde:** Implemented `CacheEngine` core logic, atomic CAS operations, and invariant checks.
- **Nikhil Shelar:** Built unit tests for version validation and lazy TTL read expiration.
- **Sanchit Gaikwad:** Integrated state validation hooks and cache configuration constraint validators.

Key Technical Deliverable: Thread-Safe `CacheEngine.java`, CAS logic, and invariant report generator.

Evaluation Status: **COMPLETED & VERIFIED**

WEEK 5 (25 Aug 2026 – 31 Aug 2026) — Phase II: Persistence & Write-Ahead Log

Core Objective & Focus: Append-Only WAL v2, CRC32 Checksums, Fsync Modes & Process Locking

Detailed Work Undertaken & Key Technical Achievements:

- Implemented `WriteAheadLog.java` supporting sequential binary serialization of `PUT` and `DELETE` operations.
- Integrated `ChecksumUtil.java` using IEEE 802.3 32-bit Cyclic Redundancy Checks (CRC32) per log record.
- Added configurable `DurabilityMode` (`OS_FLUSH` for throughput vs. `FSYNC` for hardware-level durability).
- Implemented OS-level exclusive file lock (`FileChannel.tryLock()`) on `wal.lock` to prevent split-brain dual writers.
- Added Unicode support and binary payload serialization with byte-length prefixes.
- Created stress tests validating sequential append performance under high write loads.

Individual Team Member Contributions:

- **Jay Shinde:** Optimized buffer allocation and serialization codecs for UTF-8 key-value strings.
- **Nikhil Shelar:** Created test suites validating writer exclusion when multiple JVMs access the same WAL.
- **Sanchit Gaikwad:** Implemented WAL append methods, CRC32 polynomial computation, and `fsync` control.

Key Technical Deliverable: `WriteAheadLog.java`, `ChecksumUtil.java`, and exclusive writer locking tests.

Evaluation Status: **COMPLETED & VERIFIED**

WEEK 6 (01 Sep 2026 – 07 Sep 2026) — Phase II Review & Crash Recovery Engine

Core Objective & Focus: RecoveryManager Replay State Machine, CRC Validation & Phase-II Review

Detailed Work Undertaken & Key Technical Achievements:

- Implemented `RecoveryManager.java` for deterministic reconstruction of cache state on startup.
- Built sequential log parser validating record sequence monotonicity and CRC32 integrity.
- Implemented corruption containment: invalid CRC or torn writes halt replay safely without crashing JVM.
- Implemented `RecoveryReport.java` detailing total records read, applied entries, and corrupted byte offsets.
- Conducted Phase-II milestone evaluation demonstrating crash persistence via `make verify`.
- Verified that acknowledged writes survive simulated process `SIGKILL` termination.

Individual Team Member Contributions:

- **Jay Shinde:** Integrated recovery state population into `CacheEngine` memory maps.
- **Nikhil Shelar:** Created crash-recovery integration tests (`standalone restart recovery`).
- **Sanchit Gaikwad:** Implemented `RecoveryManager` parser, CRC validation, and corruption isolation logic.

Key Technical Deliverable: `RecoveryManager.java`, `RecoveryReport.java`, and Phase-II Progress Sign-off.	**Evaluation Status: COMPLETED & VERIFIED**

WEEK 7 (08 Sep 2026 – 14 Sep 2026) — Phase III: Event-Driven Bus & Observability

Core Objective & Focus: Ordered Event Bus, Immutable Domain Events & Circular Event Journal

Detailed Work Undertaken & Key Technical Achievements:

- Implemented `EventBus.java` providing in-memory publish-subscribe domain event broadcasting.
- Defined immutable `CacheEvent.java` carrying event type, sequence ID, timestamp, key, and payload.
- Supported 7 discrete domain event types: `CACHE_PUT`, `CACHE_HIT`, `CACHE_MISS`, `CACHE_DELETE`, `TTL_EXPIRED`, `EVICTION`, `RECOVERY`.
- Implemented subscriber failure isolation: exceptions in listeners do not break core cache mutations.
- Built `InMemoryEventJournal.java` using a circular ring buffer to store recent events for live inspection.
- Added multi-threaded event ordering tests validating FIFO event dispatching.

Individual Team Member Contributions:

- **Jay Shinde:** Connected `CacheEngine` state transitions to dispatch events through the bus.
- **Nikhil Shelar:** Designed `EventBus`, `CacheEvent`, `InMemoryEventJournal`, and failure isolation wrapper.
- **Sanchit Gaikwad:** Integrated persistence lifecycle events (`RECOVERY`, `CHECKPOINT`) into the event stream.

Key Technical Deliverable: `EventBus.java`, `CacheEvent.java`, `InMemoryEventJournal.java`, and isolation tests.	**Evaluation Status: COMPLETED & VERIFIED**

WEEK 8 (15 Sep 2026 – 21 Sep 2026) — Phase III: Telemetry Metrics & Audit Logging

Core Objective & Focus: Metrics Projection, JSON Lines Audit Sink & Real-Time Hit Ratio Tracking

Detailed Work Undertaken & Key Technical Achievements:

- Implemented `CacheMetrics.java` projecting real-time telemetry directly from event streams.
- Calculated hits, misses, hit ratio percentage, write counts, eviction totals, and operation latencies.
- Implemented `AuditLogListener.java` streaming structured JSON Lines records to append-only `audit.jsonl`.
- Built `MetricsSnapshot.java` providing immutable point-in-time telemetry captures.
- Verified that metrics update asynchronously without injecting locking overhead into critical read paths.
- Added integration tests verifying audit file output formatting and metric calculation accuracy.

Individual Team Member Contributions:

- **Jay Shinde:** Optimized thread-safe atomic accumulators for high-throughput metric updates.
- **Nikhil Shelar:** Implemented `CacheMetrics`, `MetricsSnapshot`, and `AuditLogListener` JSONL sink.
- **Sanchit Gaikwad:** Tested audit log durability and forensic reconstruction from JSONL files.

Key Technical Deliverable: `CacheMetrics.java`, `AuditLogListener.java`, and telemetry verification test cases.

Evaluation Status: **COMPLETED & VERIFIED**

WEEK 9 (22 Sep 2026 – 28 Sep 2026) — Phase III Review: TTL Sweeper & Eviction Algorithms

Core Objective & Focus: LRU & LFU Eviction Policies, Proactive TTL Cleaner & Phase-III Review

Detailed Work Undertaken & Key Technical Achievements:

- Implemented $O(1)$ `LRUCache.java` using custom doubly linked list and hash map index.
- Implemented $O(1)$ `LFUCache.java` with frequency nodes, doubly linked bucket lists, and LRU tie-breaking.
- Built proactive background TTL cleaner thread in `CacheEngine` sweeping expired keys periodically.
- Ensured evictions and TTL expirations dispatch `EVICTION` and `TTL_EXPIRED` events to Event Bus.
- Conducted Phase-III review demonstrating event streaming, metrics, and bounded eviction.
- Verified zero memory leaks under continuous synthetic write workloads.

Individual Team Member Contributions:

- **Jay Shinde:** Implemented background TTL sweeper thread and safe entry expiration removal.
- **Nikhil Shelar:** Implemented `LRUCache` and `LFUCache` data structures and eviction unit tests.
- **Sanchit Gaikwad:** Connected eviction triggers to log deletion operations to persistent WAL.

Key Technical Deliverable: `LRUCache.java`, `LFUCache.java`, TTL Sweeper, and Phase-III Progress Sign-off.

Evaluation Status: **COMPLETED & VERIFIED**

WEEK 10 (29 Sep 2026 – 05 Oct 2026) — Phase IV: Checkpointing & Log Compaction

Core Objective & Focus: SnapshotManager, Atomic State Checkpointing & WAL Compaction

Detailed Work Undertaken & Key Technical Achievements:

- Implemented `SnapshotManager.java` for writing atomic binary snapshots (`snapshot.dat`) of live cache state.
- Integrated CRC32 checksums and metadata headers into snapshot binary formats.
- Implemented WAL Compaction: once snapshot succeeds, older WAL records prior to checkpoint are safely truncated.
- Implemented hybrid recovery in `RecoveryManager`: load snapshot base -> replay subsequent WAL records.
- Built `CheckpointReport.java` tracking snapshot duration, file size, compacted bytes, and active key counts.
- Verified that snapshot-plus-WAL recovery reduces startup replay time by over 80%.

Individual Team Member Contributions:

- **Jay Shinde:** Optimized in-memory entry serialization for atomic snapshot generation.
- **Nikhil Shelar:** Added automated checkpoint verification tests and diagnostic output commands.
- **Sanchit Gaikwad:** Implemented `SnapshotManager`, WAL truncation, and snapshot-first recovery loader.

Key Technical Deliverable: `SnapshotManager.java`, `CheckpointReport.java`, and WAL compaction test suite.	**Evaluation Status:** **COMPLETED & VERIFIED**

WEEK 11 (06 Oct 2026 – 12 Oct 2026) — Phase IV: B-Tree Indexing & Transactions

Core Objective & Focus: B-Tree Range Queries, Multi-Key Atomic Transactions & Phase-IV Review

Detailed Work Undertaken & Key Technical Achievements:

- Implemented self-balancing `BTreeIndex.java` supporting lexicographical range scans (`scan(from, to)`).
- Implemented `TransactionManager.java` supporting multi-key atomic transactions (`begin`, `put`, `commit`, `rollback`).
- Enforced 2-Phase Locking (2PL) acquiring key locks in alphabetical order to eliminate deadlock risks.
- Implemented atomic WAL transaction manifests with SHA-256 batch integrity verification.
- Handled torn transaction recovery: uncommitted or damaged transaction batches are cleanly skipped.
- Completed Phase-IV review with end-to-end integration across all subsystems.

Individual Team Member Contributions:

- **Jay Shinde:** Implemented `BTreeIndex.java` node splitting/search and `TransactionManager.java` 2PL locks.
- **Nikhil Shelar:** Implemented transaction timeout reaper and isolation conflict test suites.
- **Sanchit Gaikwad:** Implemented atomic WAL transaction batch serialization and SHA-256 manifest checks.

Key Technical Deliverable: `BTreeIndex.java`, `TransactionManager.java`, and Phase-IV Progress Sign-off.	**Evaluation Status:** **COMPLETED & VERIFIED**

WEEK 12 (13 Oct 2026 – 20 Oct 2026) — Final ESE Preparation & Comprehensive Evaluation

Core Objective & Focus: End-to-End Test Suite (22 Tests), Benchmarks, Complete Documentation & ESE Defense

Detailed Work Undertaken & Key Technical Achievements:

- Unified complete module facade in `PersistentEventDrivenCache.java` with centralized lifecycle management.
- Finalized full test harness (`AllTests.java`) running 22 comprehensive tests in 0.34 seconds with 100% pass rate.
- Executed synthetic benchmarks comparing LRU vs. LFU across Uniform and Zipfian access distributions.
- Completed production CLI (`CommandHandler.java`) with `DIAGNOSTICS`, `BENCHMARK`, `JOURNAL`, and `RECOVERY`.
- Compiled final project documentation, architecture book, 50-page viva handbook, and Phase-I artifacts.
- Conducted mock viva sessions covering all 40 core systems questions for End Semester Examination (ESE).

Individual Team Member Contributions:

- **Jay Shinde:** Compiled project facade, B-Tree range query benchmarks, and concurrency verification.
- **Nikhil Shelar:** Orchestrated final test suite, benchmark percentile reporting, and demonstration scripts.
- **Sanchit Gaikwad:** Validated crash-recovery failure injection tests and finalized storage documentation.

Key Technical Deliverable: Complete Jar Package, 22/22 Passing Test Suite, Final Project Report, and ESE Presentation.

Evaluation Status: **COMPLETED & VERIFIED**

12-WEEK SEMESTER PROGRESS AT A GLANCE

Week	Dates	Phase Milestone	Key Module / Artifact Delivered	Status
WEEK 1	28 Jul 2026 – 03 Aug 2026	Phase I: Problem Definition & Inception	Draft Problem Statement, Initial Project Scope Document, and Conceptual Architecture Model.	**VERIFIED**
WEEK 2	04 Aug 2026 – 10 Aug 2026	Phase I: Literature Survey & Gap Analysis	Literature Survey Matrix, Gap Analysis Report, and WAL Specification Draft.	**VERIFIED**
WEEK 3	11 Aug 2026 – 17 Aug 2026	Phase I Review & Baseline Prototype	Phase-I Signed Report, `phase-1/` documentation repository, and baseline CLI prototype.	**VERIFIED**
WEEK 4	18 Aug 2026 – 24 Aug 2026	Phase II: Core Engine & Memory Subsystem	Thread-Safe `CacheEngine.java`, CAS logic, and invariant report generator.	**VERIFIED**
WEEK 5	25 Aug 2026 – 31 Aug 2026	Phase II: Persistence & Write-Ahead Log	`WriteAheadLog.java`, `ChecksumUtil.java`, and exclusive writer locking tests.	**VERIFIED**
WEEK 6	01 Sep 2026 – 07 Sep 2026	Phase II Review & Crash Recovery Engine	`RecoveryManager.java`, `RecoveryReport.java`, and Phase-II Progress Sign-off.	**VERIFIED**
WEEK 7	08 Sep 2026 – 14 Sep 2026	Phase III: Event-Driven Bus & Observability	`EventBus.java`, `CacheEvent.java`, `InMemoryEventJournal.java`, and isolation tests.	**VERIFIED**
WEEK 8	15 Sep 2026 – 21 Sep 2026	Phase III: Telemetry Metrics & Audit Logging	`CacheMetrics.java`, `AuditLogListener.java`, and telemetry verification test cases.	**VERIFIED**
WEEK 9	22 Sep 2026 – 28 Sep 2026	Phase III Review: TTL Sweeper & Eviction Algorithms	`LRUCache.java`, `LFUCache.java`, TTL Sweeper, and Phase-III Progress Sign-off.	**VERIFIED**
WEEK 10	29 Sep 2026 – 05 Oct 2026	Phase IV: Checkpointing & Log Compaction	`SnapshotManager.java`, `CheckpointReport.java`, and WAL compaction test suite.	**VERIFIED**
WEEK 11	06 Oct 2026 – 12 Oct 2026	Phase IV: B-Tree Indexing & Transactions	`BTreeIndex.java`, `TransactionManager.java`, and Phase-IV Progress Sign-off.	**VERIFIED**
WEEK 12	13 Oct 2026 – 20 Oct 2026	Final ESE Preparation & Comprehensive Evaluation	Complete Jar Package, 22/22 Passing Test Suite, Final Project Report, and ESE Presentation.	**VERIFIED**

Prepared by Project Team:
1. Jay Shinde (2024800115)
2. Nikhil Shelar (2024800113)
3. Sanchit Gaikwad (2024800029)

Reviewed & Approved by:

Prof. Suhas Kakade
Project Guide & Supervisor
Department of CSE, S.P.I.T.